ASSIGNMENT 01

Git, Gitea, GitHub, Git LFS, and GitHub Pages

Student Name	Munazza Ishfaq Allies Shaista
Registration Number	Fa24b1-se-056
Course / Section	Cloud Computing-B
Submission Repository	CC → Assignments/Assignment01

Assignment Objective

The goal is to run Gitea on the Ubuntu server, create one local Git repository and push it to both Gitea and GitHub, use Git LFS for three large files, and publish a personal portfolio/CV using GitHub Pages.

Submission structure

```
CC/
└── Assignments/
    └── Assignment01/
        ├── Assignment01.md
        ├── Assignment01_Solution.docx
        ├── Assignment01_Solution.pdf
        └── screenshots/
            ├── task1_gitea_running.png
            ├── task1_gitea_push.png
            ├── task1_gitea_repository.png
            ├── task2_remotes.png
            ├── task2_github_push.png
            ├── task2_github_repository.png
            ├── task3_lfs_setup.png
            ├── task3_lfs_files.png
            ├── task3_lfs_push.png
            ├── task4_pages_repository.png
            ├── task4_pages_deployment.png
            └── task4_portfolio_live.png
```

Getting Started:

Prepare the evidence folder

In your existing public GitHub repository named CC, create the assignment folder and screenshots subfolder. If the CC repository is already cloned on the Ubuntu server, run the following from that repository's root directory:

```
mkdir -p Assignments/Assignment01/screenshots
```

If CC is not cloned on the Ubuntu server, you can create the screenshots folder in your local copy and later commit and push the completed assignment folder to CC. Do not create a second repository named CC if you already have one.

Task 1 — Install Gitea and push a repository from Ubuntu

1.1 Install Docker and Docker Compose

Step 1: Update packages and install Docker Engine and curl.

```
sudo apt update -y
sudo apt install -y docker.io curl
```

Step 2: Install the Docker Compose CLI plug-in.

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-
x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

Step 3: Start Docker and add your account to the docker group.

```
sudo systemctl start docker
sudo usermod -aG docker $USER
exit
```

After exit, completely disconnect/sign out of the Ubuntu server and reconnect. Opening only another terminal tab does not refresh group membership.

Step 4: Reconnect and verify the tools.

```
ssh <munazzaishfaqalliesshaista056>@<192.168.142.130>
git --version
docker --version
docker compose version
```

Do not continue until all three version commands return version information. If you already have a terminal directly on the assigned Ubuntu server, reconnect using your instructor's prescribed method.

Step 5: Clone the instructor's Gitea setup repository.

```
cd ~
git clone https://github.com/WaqasSaleem97/Gitea.git
cd Gitea
```

Step 6: Start Gitea and PostgreSQL with the supplied compose.yaml.

```
docker compose up -d
```

Step 7: Check that both services are running.

```
docker compose ps
```

Step 8: Check Gitea's local HTTP response.

```
curl -sS -o /dev/null -w "Gitea HTTP status: %{http_code}\n" http://192.168.142.130:3000/
```

A successful HTTP response is normally 200. If the containers are still starting, wait briefly and repeat the checks.

Step 9: Find the server IP address.

```
ip addr
```

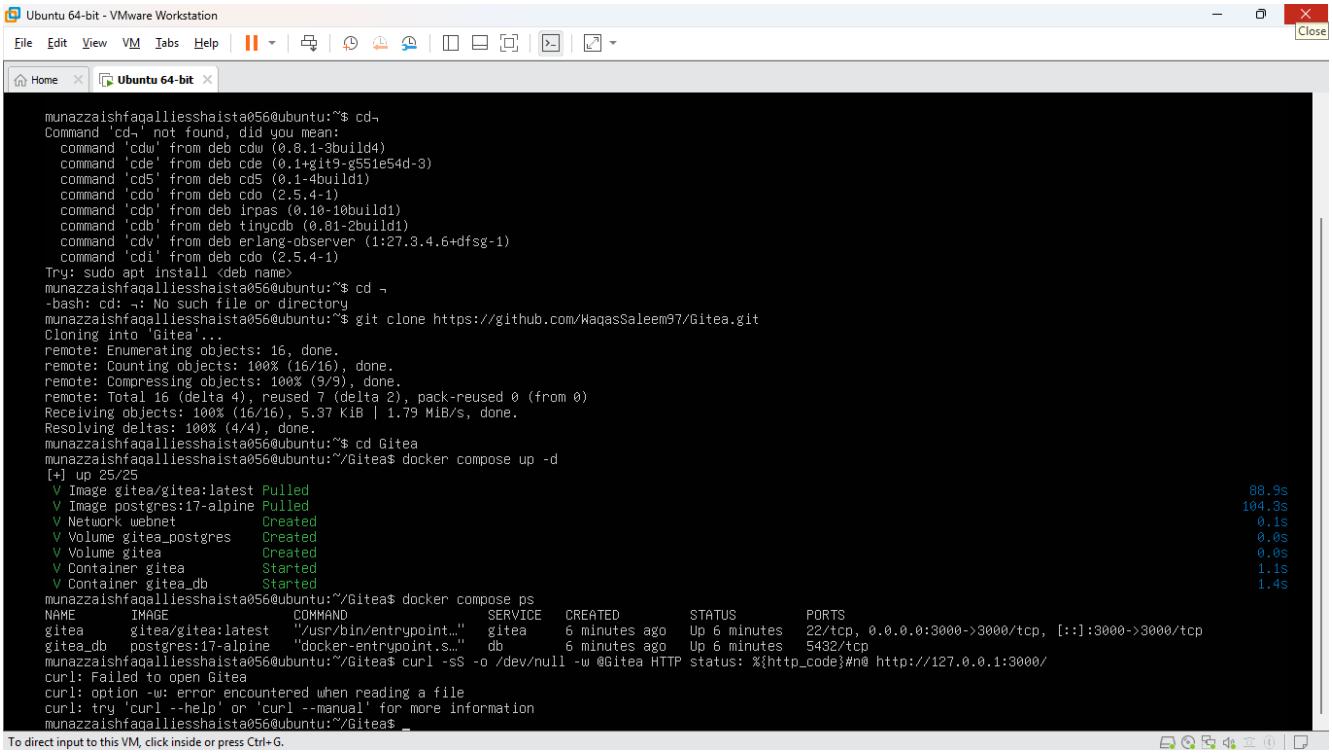
Step 10: Open Gitea in a browser.

Visit http://192.168.114.130:3000/ using the IP address.

1.1.1 Complete the first-time Gitea setup

Setting	Value
Database type	PostgreSQL
Database host	db:5432
Database username	gitea
Database password	gitea
Database name	gitea
Server domain	<192.168.114.130>
Gitea base URL	http://< 192.168.114.130>:3000/

Complete the installation and create your Gitea administrator account. Do not show the administrator password or any token in screenshots.



1.2 Create the Gitea repository and push the first commit

Step 1: Create an empty public repository in Gitea.

In Gitea, create a repository named exactly Assignment01. Do not add a README, .gitignore, or license during creation.

Step 2: Create a separate local repository on the Ubuntu server.

```
cd ~
mkdir Assignment01
cd Assignment01
git init
git branch -M main
```

This local Assignment01 directory must be outside both the Gitea setup directory and your existing CC repository.

Step 3: Create README.md.

The README must contain your full name and registration number. Replace the example details below with your own information.

```
printf "# Assignment01\n\nName: Munazza Ishfaq Allies Shaista\nRegistration Number: 2024-BSE-
056\n" > README.md
cat README.md
```

Step 4: Stage and commit the README.

```
git add README.md
git commit -m "Add student information"
```

If Git asks you to configure your identity, set your own name and email, then repeat the commit:

```
git config --global user.name "Munazza Ishfaq Allies Shaista"
git config --global user.email "munazza@gmail.com"
git commit -m "Add student information"
```

Step 5: Generate a Gitea access token.

In Gitea, open your profile menu → Settings → Applications → Manage Access Tokens. Use token name ubuntu-server-token and grant read/write repository permission (write:repository). Copy the token securely.

Step 6: Add Gitea as a remote without embedding the token.

```
git remote add gitea http://<gitea-username>@127.0.0.1:3000/<gitea-username>/Assignment01.git
git remote -v
```

The remote URL should include your Gitea username but must not include a password or token.

Step 7: Push the main branch to Gitea.

```
git push -u gitea main
```

When the password prompt appears, paste the Gitea personal access token and press Enter. Verify that the push completes successfully.

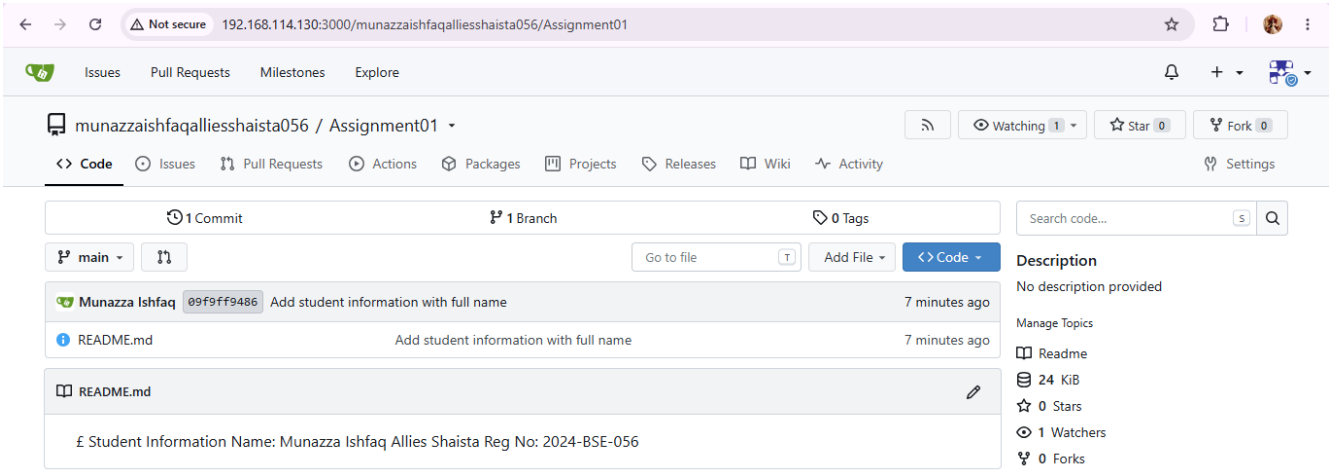
```
to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'munazzaishfaqalliesshaista056@ubuntu.(none)')
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote add gitea http://munazzaishfaqalliesshaista056"127.0.0.1:3000/munazzaishfaqalliesshaista05
6/Assignment01.git
>
> ^[S^C
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git config --global user.email 'munazza"gmail.com'

git config --global user.name 'munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$
za Ishfaq'munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git config --global user.name 'Munazza Ishfaq'
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git config --global user.name 'Munazza Ishfaq'
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git commit -m 'Add student information with full name'
[main (root-commit) 09f9ff9] Add student information with full name
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote add gitea 'http://munazzaishfaqalliesshaista056"127.0.0.1:3000/munazzaishfaqalliesshaista0
56/Assignment01.git'
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git push -u gitea main
fatal: unable to access 'http://munazzaishfaqalliesshaista056"127.0.0.1:3000/munazzaishfaqalliesshaista056/Assignment01.git/': URL rejected: Bad hostname
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote remove gitea
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote add gitea 'http://munazzaishfaqalliesshaista056"127.0.0.1:3000/munazzaishfaqalliesshaista0
56/Assignment01.git'
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git push -u gitea main
fatal: unable to access 'http://munazzaishfaqalliesshaista056"127.0.0.1:3000/munazzaishfaqalliesshaista056/Assignment01.git/': URL rejected: Bad hostname
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote remove gitea
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote add gitea http://127.0.0.1:3000/munazzaishfaqalliesshaista056/Assignment01.git
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git push -u gitea main
Username for 'http://127.0.0.1:3000': munazzaishfaqalliesshaista056
Password for 'http://munazzaishfaqalliesshaista056@127.0.0.1:3000':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 78.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/munazzaishfaqalliesshaista056/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'gitea/main'.
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$
```

1.3 Verify the Gitea repository

- Open the Assignment01 repository in Gitea.
- Confirm the owner and repository name are visible.
- Confirm that the repository is public.
- Confirm README.md displays your full name and registration number.
- Keep the Gitea URL visible in the browser.



Task 2 — Push the same repository to GitHub

2.1 Create GitHub repository and add a second remote

Step 1: Create an empty public GitHub repository.

On GitHub, create a public repository named exactly Assignment01. Do not initialize it with a README, .gitignore, or license.

Step 2: Return to the same local repository on Ubuntu and add GitHub.

```
cd ~/Assignment01
git remote add github https://github.com/<munazzaishfaqalliesshaista056>/Assignment01.git
```

Step 3: Verify both remotes.

```
git remote -v
```

The output must list both gitea and github, each with fetch and push URLs. Neither URL should contain credentials.

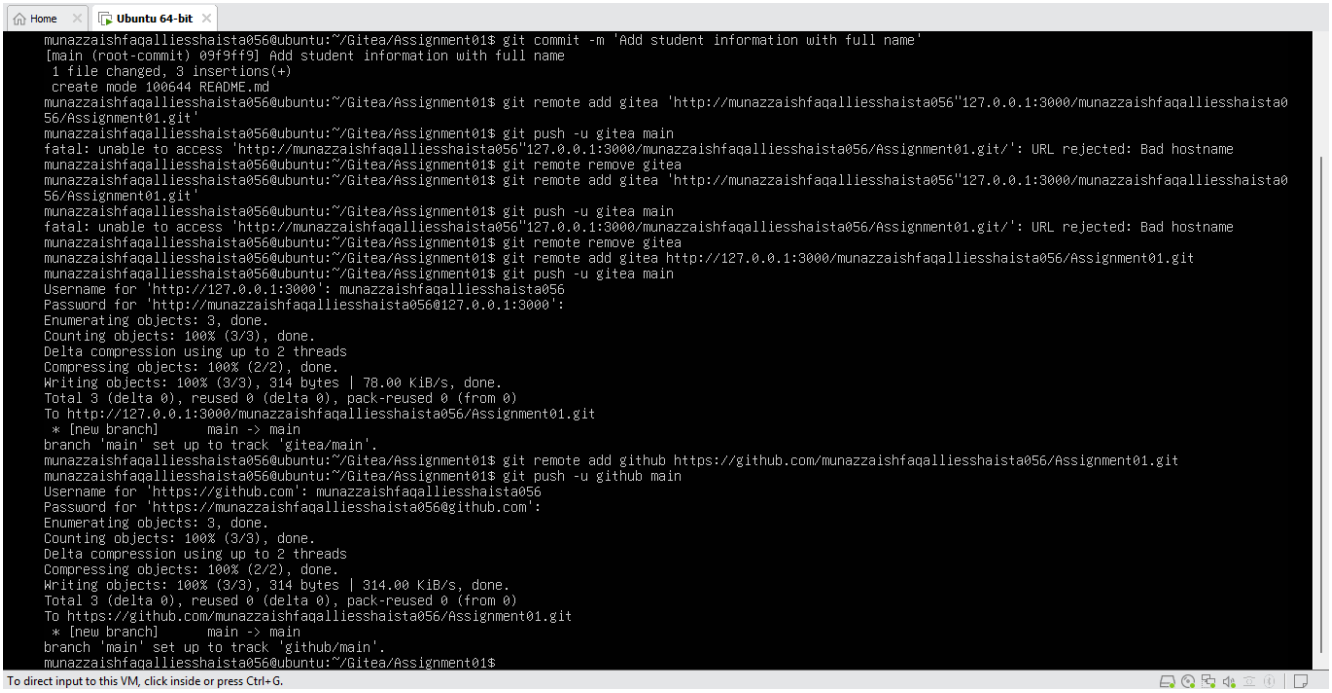
```
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$ git remote -v
gitea   http://127.0.0.1:3000/munazzaishfaqalliesshaista056/Assignment01.git (fetch)
gitea   http://127.0.0.1:3000/munazzaishfaqalliesshaista056/Assignment01.git (push)
github  https://github.com/munazzaishfaqalliesshaista056/Assignment01.git (fetch)
github  https://github.com/munazzaishfaqalliesshaista056/Assignment01.git (push)
munazzaishfaqalliesshaista056@ubuntu:~/Gitea/Assignment01$
```

2.2 Push to GitHub

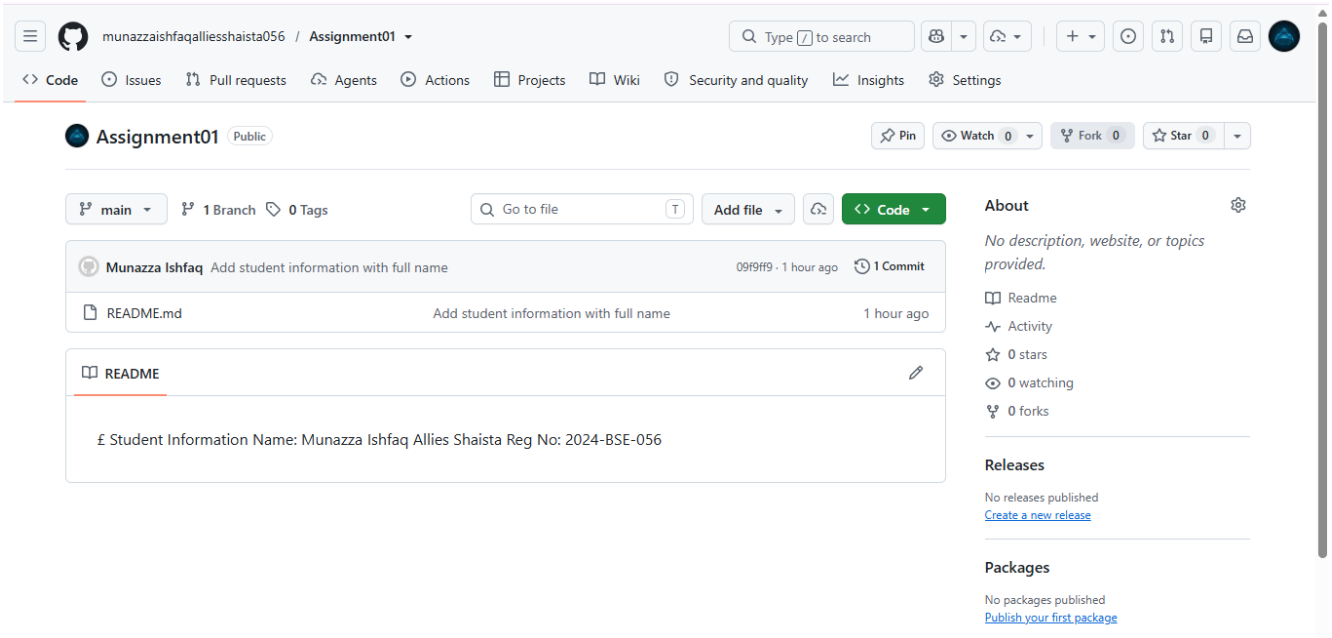
Step 1: Push main to GitHub.

```
git push -u github main
```

If GitHub asks you to authenticate over HTTPS, use the authentication method supported by your account. Never paste a token into the URL.



2.3 Verify the GitHub repository



Task 3 — Track three large files with Git LFS

3.1 Initialize Git LFS and track .bin files

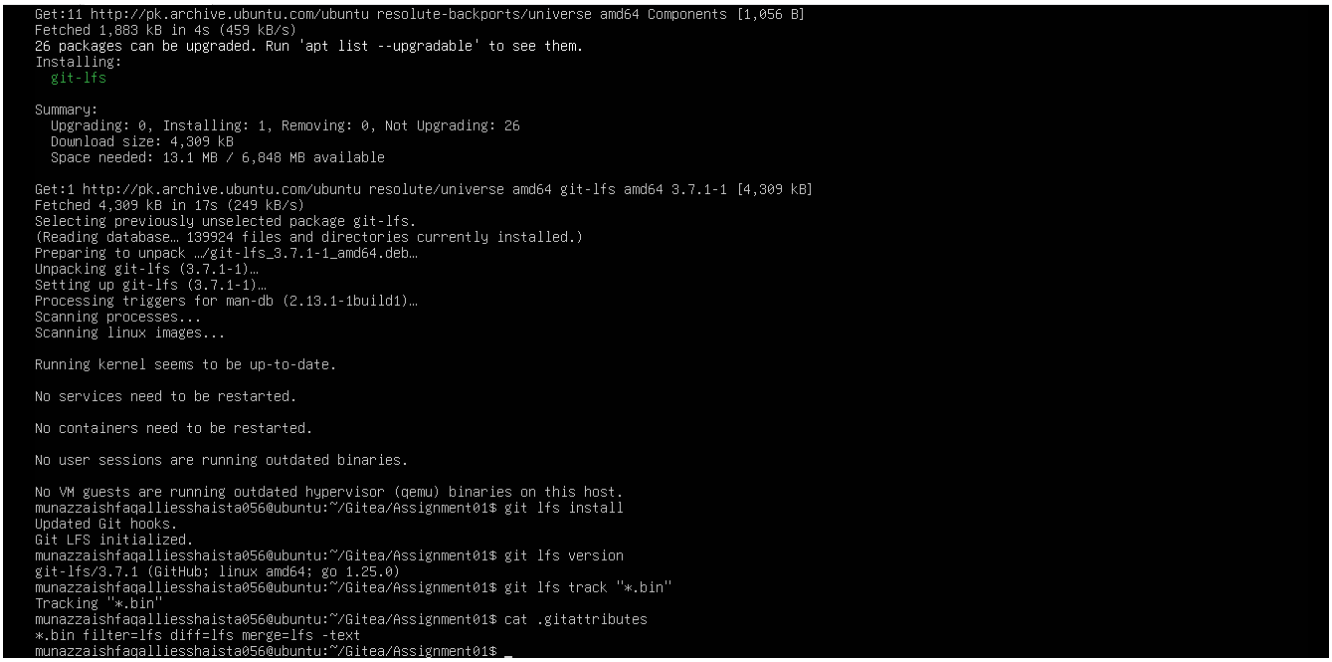
Step 1: Open the same local repository.

```
cd ~/Assignment01
```

Step 2: Initialize Git LFS, check its version, and track .bin files.

```
git lfs install
git lfs version
git lfs track "*.bin"
cat .gitattributes
```

The .gitattributes output should include a rule for *.bin. If Git LFS is not installed, install it using your course's approved instructions before continuing.



3.2 Create three files larger than 100 MiB

Step 1: Create the three files.

```
truncate -s 101M large-file-1.bin
truncate -s 102M large-file-2.bin
truncate -s 103M large-file-3.bin
```

These commands create files with the required logical sizes. Run the next commands from ~/Assignment01.

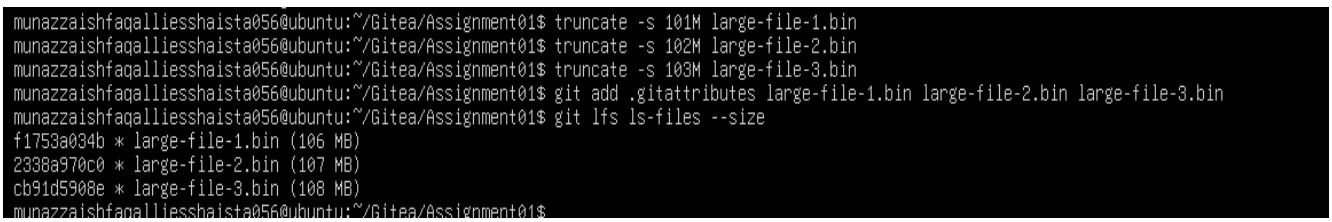
Step 2: Stage .gitattributes and all three files.

```
git add .gitattributes large-file-1.bin large-file-2.bin large-file-3.bin
```

Step 3: Verify LFS tracking and sizes.

```
git lfs ls-files --size
```

The output must list all three .bin files and show that each is larger than 100 MiB. If the files do not appear, check that git lfs track ran before git add; then stage the files again.



3.3 Commit and push the LFS files

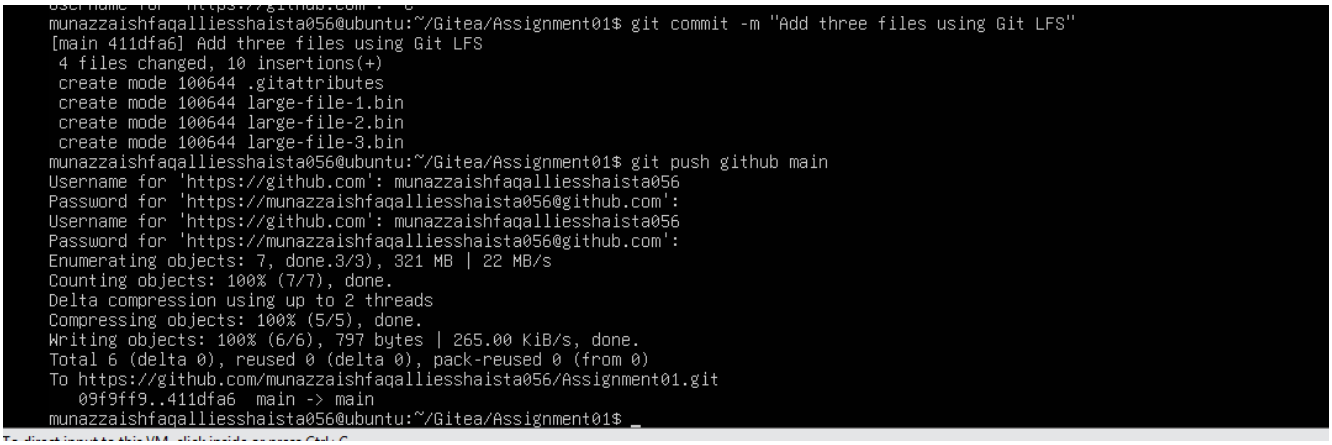
Step 1: Commit the staged files.

```
git commit -m "Add three files using Git LFS"
```

Step 2: Push to the GitHub remote.

```
git push github main
```

Wait for the LFS object upload to finish. The evidence should show a successful upload, ideally including 100% (3/3).



Task 4 — Create a portfolio/CV with GitHub Pages

4.1 Create the portfolio repository and website files

Step 1: Create a public GitHub repository.

Name it exactly munazzaishfaqalliesshaista056.github.io. This is the user-site repository name.

Step 2: Create index.html.

Create a file named index.html with your portfolio content. It must visibly contain About Me, Education, Skills, and Projects.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Portfolio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Munazza Ishfaq Allies Shaista</h1>
    <p>Student | Aspiring Professional</p>
  </header>
```

```
  <main>
    <section>
      <h2>About Me</h2>
      <p>I am a computer science student passionate about software development and version control systems.</p>
    </section>
    <section>
      <h2>Education</h2>
      <p>BS Software Engineering - Reg No: 2024-BSE-056</p>
    </section>
    <section>
      <h2>Skills</h2>
      <p>Git, GitHub, Gitea, HTML, CSS, Linux Command Line.</p>
    </section>
    <section>
      <h2>Projects</h2>
      <p>Version Control Systems Assignment & Static Portfolio Website Setup.</p>
    </section>
  </main>
  <footer>
    <p>© Munazza Ishfaq Allies Shaista</p>
  </footer>
</body>
</html>
```

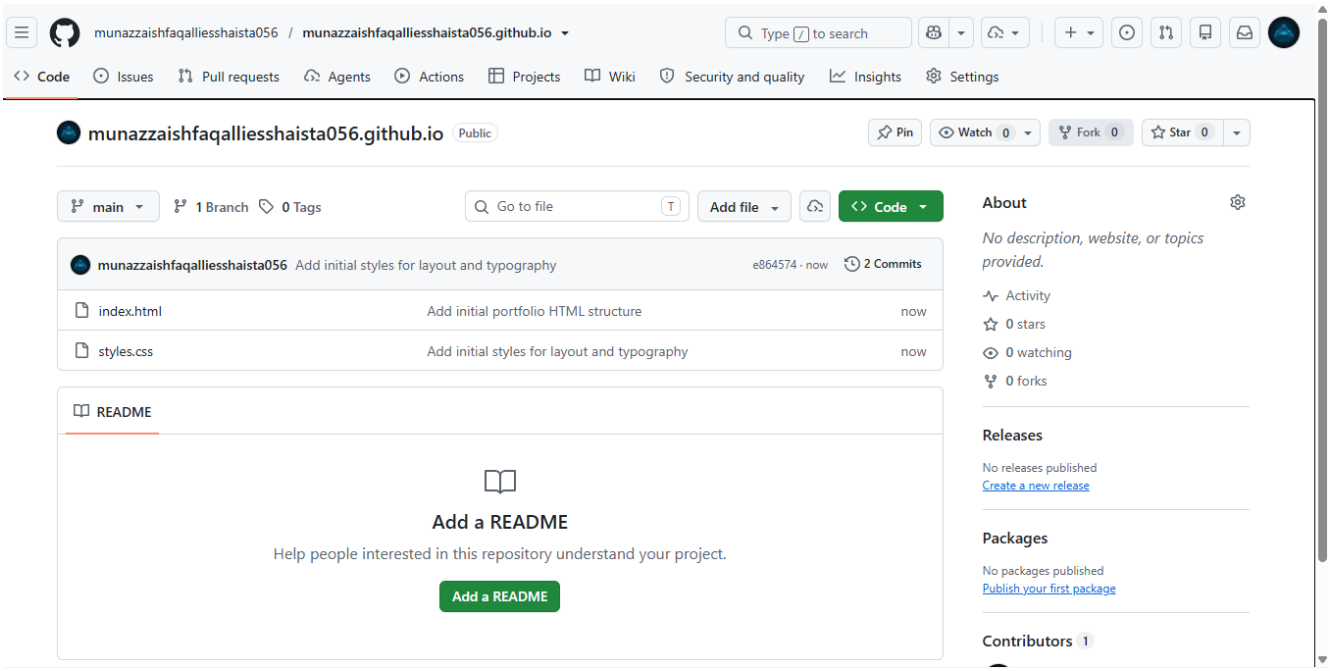
Step 3: Create styles.css.

```
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.6;
  color: #243447;
  background: #f4f7fb;
}
header {
  text-align: center;
  padding: 36px 16px;
  color: white;
  background: #17365d;
}
main {
  max-width: 850px;
  margin: 24px auto;
  padding: 0 16px;
}
section {
  background: white;
  margin-bottom: 16px;
  padding: 20px;
  border-radius: 10px;
}
h2 { color: #2874a6; }
footer { text-align: center; padding: 18px; }
```

Step 4: Commit and push the files.

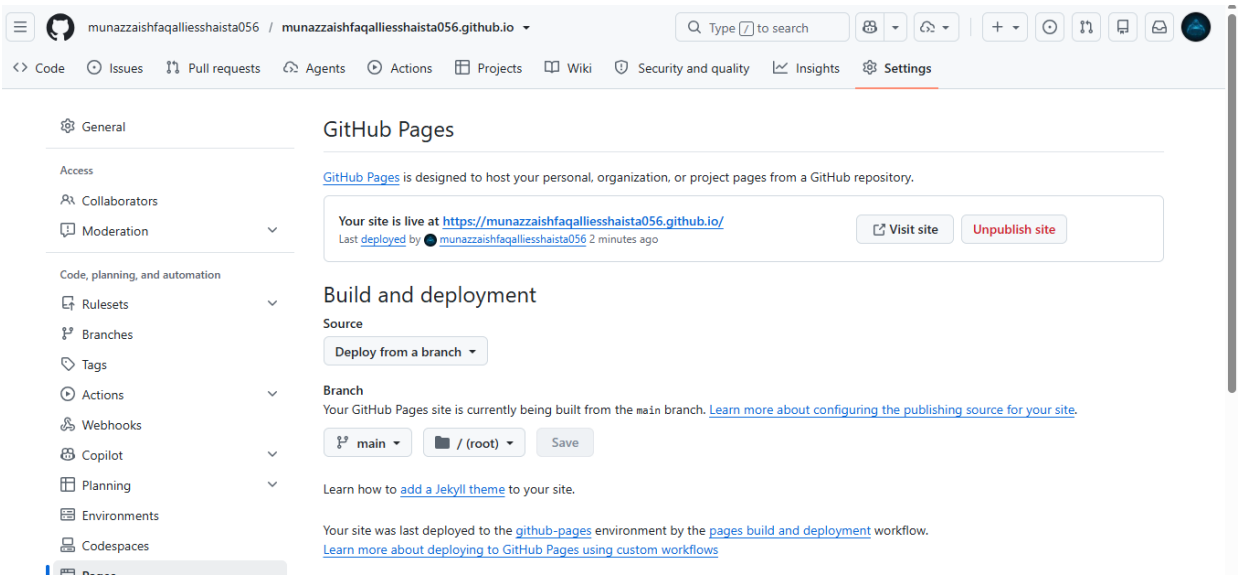
In the local folder where you created the website, initialize Git if needed, make sure the branch is main, and connect it to the new repository. If the repository is empty, a typical command sequence is:

```
git init
git branch -M main
git add index.html styles.css
git commit -m "Create portfolio website"
git remote add origin https://github.com/<your-github-username>/<your-github-
username>.github.io.git
git push -u origin main
```



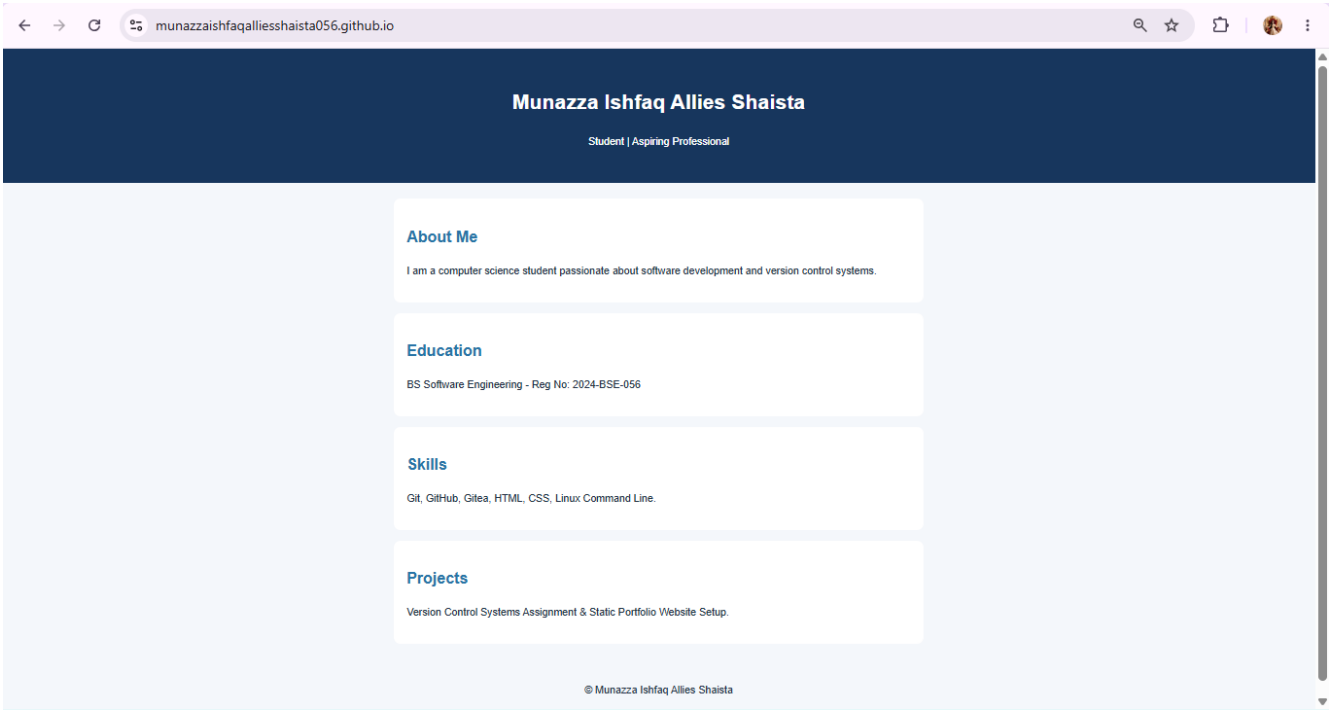
4.2 Enable GitHub Pages

- Open the portfolio repository on GitHub.
- Go to Settings → Pages.
- Under Build and deployment, select Deploy from a branch.
- Choose branch main and folder / (root), then click Save.
- Wait until GitHub shows the site as published/live and displays the site URL



4.3 Verify the live portfolio

Open https://munazzaishfaqalliesshaista056.github.io/ in browser. Confirm that the website loads and shows your portfolio content. If it is not live immediately, wait a few minutes and refresh the page.



Final checklist

- ☐ Assignment01.md is in CC/Assignments/Assignment01/.
- ☐ Assignment01_Solution.docx is in CC/Assignments/Assignment01/ and opens correctly.
- ☐ Assignment01_Solution.pdf is in the same folder and matches the Word solution.
- ☐ The screenshots folder contains all 12 required screenshots with the exact filenames.
- ☐ Screenshots show my own work and are readable.
- ☐ My GitHub username and ubuntu hostname are visible in terminal screenshots.
- ☐ No screenshot or file exposes passwords, tokens, private keys, or other credentials.
- ☐ The CC, GitHub Assignment01, and <username>.github.io repositories are public.
- ☐ The Gitea Assignment01 repository is public and contains the README.
- ☐ Both gitea and github remotes are configured in the same local repository.
- ☐ All three large files are tracked by Git LFS and successfully pushed to GitHub.
- ☐ The live GitHub Pages site opens and shows the required portfolio sections.
- ☐ The completed assignment folder has been committed and pushed to the existing CC repository.
- ☐ The Word solution has been exported to PDF, and both files have been checked.
- ☐ The three required links have been submitted on Google Classroom, as instructed.

Required links to submit

Fill in these links after completing the work:

https://github.com/munazzaishfaqalliesshaista056/CC/tree/main/Assignments/Assignment01/screenshots

- GitHub Assignment01 repository: https://github.com/munazzaishfaqalliesshaista056/Assignment01

- Live portfolio: https://munazzaishfaqalliesshaista056.github.io/

Conclusion

By completing these tasks, I practiced running a self-hosted Git service, managing a local repository with multiple remotes, pushing source code to GitHub, tracking large files with Git LFS, and publishing a static portfolio through GitHub Pages. The screenshots attached to this report provide evidence of the commands, repository contents, and deployment results.